

ToddySoft Connect ctrlX Driver - Ignition 8.1

Ignition 8.1 module for connecting to Bosch Rexroth **ctrlX CORE** controllers over the ctrlX Data Layer's HTTPS REST API. Every node the Data Layer exposes is reachable the same way — PLC symbols, axis state, framework metrics — addressed by its slash-separated node path.

This is an HTTPS management interface, not a fieldbus. Each tag in a read costs one HTTP request, so this driver suits diagnostics, configuration and moderate-rate supervision rather than high-rate process data. Size your tag groups accordingly; see *Tag Groups*.

Installation

This module is for Ignition 8.1 only. It is built against the Ignition 8.1 SDK and will not run on any other major version. For Ignition 8.3, install the separate `...-8.3.mod1` build instead — the two are not interchangeable, and each gateway needs the build that matches it.

Ignition 8.0 and earlier are not supported and cannot be. The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: ...opcua.server.api.AbstractDeviceModuleHook`.

Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.

The ToddySoft Connect modules are signed with a self-signed certificate (issued to *ToddySoft GmbH*) rather than a publicly accredited code-signing certificate. Ignition therefore asks you to review and accept that certificate once during installation - it is then remembered for all future ToddySoft modules signed with the same certificate.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**. Scroll to the bottom and click **Install or Upgrade a Module...**

2. Choose the `ToddySoft-Connect-CtrlX-Driver-8.1.modl` file and click **Install**.
3. Ignition shows the self-signed certificate details (subject and issuer *ToddySoft GmbH*) together with a warning that the module was not written by Inductive Automation. Review the certificate, tick **I understand the risk and want to install this module** and click **Add Certificate and Install Module**.
4. The module is now trusted and listed under the **ToddySoft GmbH (self-signed certificate)** group with the state **Running**.

ToddySoft GmbH (self-signed certificate) View Certificate					
Name	Version	Description	License	State	
ToddySoft Connect CtrlX Driver	1.3.0-SNAPSHOT (b0)	ToddySoft Ignition driver for Bosch Rexroth CtrlX protocol. Built for Ignition 8.1.	Trial	Running	More ▾ restart
ToddySoft Connect S7 Driver	1.3.0-SNAPSHOT (b0)	ToddySoft Ignition driver for Siemens S7 protocol. Built for Ignition 8.1.	Trial	Running	More ▾ restart
ToddySoft Connect S7 Web API Driver	1.3.0-SNAPSHOT (b0)	ToddySoft Ignition driver for the Siemens SIMATIC Web API. A management-plane path for diagnostics, alarms and operating mode; use the S7 module for process data. Built for Ignition 8.1.	Trial	Running	More ▾ restart

Before you start: the controller side

Two things must be true on the ctrlX CORE before a connection can succeed.

- **An account with the right role.** ctrlX manages read and write as separate permissions, so an account that browses and reads happily may still refuse every write with HTTP 403. Roles are assigned in the controller's own user management.
- **The Data Layer service running**, and HTTPS reachable on port 443 (or whatever you override the port to).

The certificate a ctrlX CORE ships with is self-signed and chains to no public CA, so ordinary verification cannot pass. See *Reaching a device with its factory certificate* below — the answer is to pin it, not to switch verification off and leave it off.

Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.

Name	Type	Description	Enabled	Status	
Sorting-Lane-S7	ToddySoft Connect S7 Driver	S7-1200 on the sorting lane	true	Connected	delete edit
Sorting-Lane-WebAPI	ToddySoft Connect S7 Web API Driver	S7-1200 on the sorting lane	true	Connected	delete edit

[→ Create new Device...](#)

2. Click **Create new Device...**, select **ToddySoft Connect CtrlX Driver** as the device type and click **Next**.

☐ **Siemens S7-400**
Connect to Siemens S7-400 PLCs over Ethernet.

☐ **TCP Driver**

☒ **ToddySoft Connect CtrlX Driver**
ToddySoft Connect driver for Bosch Rexroth CtrlX protocol

3. Configure the connection settings (see the sections below) and click **Save**.

4. The new device then appears in the Device Connections list.

Name	Type	Description	Enabled	Status	
CtrlX-Bench	ToddySoft Connect CtrlX Driver	Bosch Rexroth ctrlx on the bench	true	Connected	<button>delete</button> <button>edit</button>
Sorting-Lane-S7	ToddySoft Connect S7 Driver	S7-1200 on the sorting lane	true	Connected	<button>delete</button> <button>edit</button>
Sorting-Lane-WebAPI	ToddySoft Connect S7 Web API Driver	S7-1200 on the sorting lane	true	Connected	<button>delete</button> <button>edit</button>

[→ Create new Device...](#)

Ignition 8.1 has no device discovery, for this or any other driver — its device page offers no such action. Enter the host by hand. The Ignition 8.3 module *does* find ctrlX devices, by asking over mDNS and then confirming each responder; if that matters to you, it is a reason to run the 8.3 module rather than something that can be added here. Neither version offers a *Test Connection* action for ctrlX: create the device and read its status.

Setting blocks

The configuration is organized into categories: **General**, **Transport Settings**, **CtrlX Settings** and **Audit Log Settings**.

General

Assigns a name (and optional description) to the connection and controls whether it is enabled.

General	
Name	CtrlX-Bench
Description	Bosch Rexroth ctrlx on the bench
Enabled	☒ (default: true)

Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	true

This is the name the connection is then referenced under in other parts of Ignition.

Transport Settings

Transport Settings	
Host	192.168.42.100 Hostname or IP address of the CtrlX device
Port	443 Port number (default: 443) (default: 443)

Setting	Description	Default
Host	Hostname or IP address of the ctrlX device	
Port	HTTPS port of the Data Layer REST API	443

There is no transport selector, and none is missing: this driver speaks HTTPS and nothing else. The port is a plain override for a device behind a reverse proxy or a non-standard listener.

CtrlX Settings

CtrlX Settings	
Username	boschrexroth Username for authentication with the CtrlX device
Password	***** Password for authentication with the CtrlX device
Verify SSL	☐ Whether to verify SSL certificates (default: false) (default: false)
Base Path	/automation/api/v2 Base path for the REST API (default: /automation/api/v2) (default: /automation/api/v2)
Request Timeout	5000 Default timeout for requests in milliseconds (default: 5000) (default: 5.000)
Pinned Certificate Fingerprints	 Comma-separated SHA-256 fingerprints of the leaf certificates to accept (hex, colons optional). When set, only these certificates are accepted and Verify SSL is overridden - the secure way to reach a ctrlX device with its factory self-signed certificate, which chains to no public CA. The connect log prints the device's fingerprint when Verify SSL is off, ready to copy
Max Browse Depth	20 Maximum nesting depth a ** wildcard descends. Each level costs one HTTP browse per node, so an unexpectedly deep or self-referential tree spends requests rather than stack. Real ctrlX Data Layer trees are a handful of levels deep (default: 20) (default: 20)
Browse Queries	/plc/app/Application/sym/PLC_PRG Query string for symbol browsing (default: "/plc/app/Application/sym/PLC_PRG"). Separate multiple queries with ";" (default: /plc/app/Application/sym/PLC_PRG)

Setting	Description	Default
Username	Account the driver authenticates as	
Password	That account's password	

Setting	Description	Default
Verify SSL	Whether the device certificate is verified	false
Base Path	Base path of the REST API	/automation/api/v2
Request Timeout	Milliseconds allowed for one HTTP request	5000
Pinned Certificate Fingerprints	SHA-256 fingerprints of the exact leaf certificates to accept	
Max Browse Depth	How far a ** wildcard descends	20
Browse Queries	Data Layer path(s) to browse for symbols; separate multiple queries with ;	/plc/app/Application/sym/PLC_PRG

Username and password are both required, and the form says so. ctrlX

authenticates every request and the driver refuses an unauthenticated connection, so leaving either empty is rejected when you save, naming the field. They are *not* part of the connection string: the driver takes them as a credential object, which is why they never appear in a logged URL.

Reaching a device with its factory certificate

A ctrlX CORE's factory certificate is self-signed, so it chains to no public CA and names the device rather than its address. Ordinary verification cannot pass, and since Connect 1.2.0 **Verify SSL** also checks the hostname — which such a certificate fails on both counts.

Pin it. Put the device's SHA-256 leaf fingerprint in **Pinned Certificate Fingerprints** (hex, colons optional; comma-separate several). A pin accepts exactly the certificate you listed and refuses a substituted one even though that one is equally self-signed, which is strictly stronger than turning verification off. Pinning **overrides Verify SSL**, and the hostname is deliberately not checked in that mode — the pin already identifies the peer.

To obtain the fingerprint, connect once with **Verify SSL** off: the driver logs the device's SHA-256 ready to copy. Then paste it into the field. Where your site runs its own CA, provisioning a real certificate remains the better answer.

Verify SSL off accepts any peer at all, which is why it is a starting point for getting the fingerprint and not a resting place.

Max Browse Depth

Each level a `**` wildcard descends costs one HTTP browse per node, so an unexpectedly deep — or self-referential — subtree spends requests rather than stack. Real Data Layer trees are a handful of levels deep, and the default of `20` is far above anything legitimate; it is a backstop, not a tuning knob.

Audit Log Settings

Audit Log Settings

Audit Log File

Path to the audit log file where events will be recorded. Leave empty to disable audit logging.

Setting	Description	Default
Audit Log File	Path to a file protocol audit events are written to; empty disables	

The audit log records protocol-level events and grows quickly, so enable it only while investigating a problem.

Tag Addressing

ctrlX addresses are Data Layer node paths — slash-separated, like a filesystem, **with no leading slash**:

```
<segment>(/<segment>)*
```

Address	Meaning
<code>plc/app/Application/sym/PLC_PRG/myVariable</code>	A PLC variable exposed through the Data Layer
<code>motion/axs/MyAxis/state/values/actual/pos</code>	An axis's actual position
<code>framework/metrics/system/cpu-utilisation-percent</code>	A single CPU-utilisation node
<code>framework/metrics/system/**</code>	Pattern — every descendant of that namespace (browsing only)

A leading `/` or stray whitespace is the usual cause of `INVALID_ADDRESS` from a read or write. Note the asymmetry with **Browse Queries**, whose default *does* begin with `/`: that field is a browse path, not a tag address.

The Data Layer tree is large. Restrict **Browse Queries** to the subtree you actually want — the default, `/plc/app/Application/sym/PLC_PRG`, lists the symbols of one PLC program — rather than browsing from the root.

Types

Data Layer nodes carry their type in the response payload (`int8`, `uint16`, `float64`, `string`, `bool`, ...), and the driver maps that to the matching OPC UA type. Nothing has to be declared in the address.

A node the controller reports without a usable type is skipped, with a `WARN` naming it, and the rest of the browse is unaffected — one untypeable node does not cost the device.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

This matters more here than on a fieldbus driver. Each tag in a read is a separate HTTP request, so a hundred tags at one second is a hundred requests a second against a management interface. Put slow-moving diagnostics in a slower group.

Troubleshooting

Device shows "Faulted" or sits in "Reconnecting":

- Verify the device is reachable over HTTPS on the configured port — `curl -k https://<host>/` is enough to tell a network problem from a certificate one
- Review Gateway logs: **Status > Diagnostics > Logs**

SSL handshake failed although the device answers:

- The factory certificate cannot pass ordinary verification. Pin its fingerprint under **Pinned Certificate Fingerprints** — see *Reaching a device with its factory certificate*

HTTP 401 Unauthorized:

- Wrong username or password, or the account was left empty

HTTP 403 Forbidden — reads work, writes are refused:

- The account authenticated, but lacks the role the operation needs. ctrlX manages read and write as separate permissions; adjust the role in the controller's user management

HTTP 404 Not Found:

- The node path does not exist. Browse to discover legal paths rather than typing them

INVALID_ADDRESS from a read or write:

- Almost always a leading `/` or whitespace in the address. Tag addresses have no leading slash

Bulk reads are slow:

- Expected: each tag issues its own HTTP call. Spread tags across slower tag groups

Docker related issues

The ctrlX protocol carries no identity for this end of the connection, so a container's internal address is never announced to the device and nothing has to be reconciled. A gateway in a bridged container connects normally, provided the device's HTTPS port is routable.

VPN related issues

A remote device reached over a VPN needs nothing special from this driver, for the same reason: the connection is an ordinary outbound HTTPS session. Only reachability and the certificate matter.

Version history

Unreleased — 1.3.0

- First release of this manual.